

OWASP A01 – Broken Access Control Case Study

Executive Summary

This case study documents the identification and validation of a Broken Access Control vulnerability aligned with **OWASP Top 10 A01**.

Using two normally authenticated user accounts, object-level authorization was evaluated on a user-owned resource endpoint. By modifying a direct object reference while preserving authentication context, cross-user access was achieved, confirming the absence of server-side ownership enforcement.

The issue was validated through both positive and negative testing, followed by a remediation analysis outlining controls required to prevent unauthorized access to user-owned resources.

Target & Tooling Overview

Target: OWASP Juice Shop v14.x (Docker)

Host: Ubuntu 24.04, Docker 29.1.5

Testing Date: 2026-01-29

Browsers: Mullvad (User A), Firefox (User B)

Authentication Method: normal registration

Actor	Auth Method	User ID	Role
User1	Normal Register	23	Customer
User2	Normal Register	24	Customer

Authenticated requests were manually replayed and modified using PowerShell to preserve session headers while validating object-level authorization behavior.

Authorization Context

Multiple authenticated users accessed the same application endpoint in isolated sessions. To evaluate object-level authorization enforcement, all request parameters were held constant except for the referenced object identifier.

This approach ensured that any unauthorized access observed could be attributed to missing server-side ownership validation rather than changes in authentication state or request structure.

Account Header Capture Information

Figure 1 & 2: Developer Tools network captures of authenticated GET requests from separate user sessions. These captures establish baseline request structure and authentication context prior to modifying only the referenced resource identifier.

OWASP A01 – Broken Access Control Case Study

Figure 1

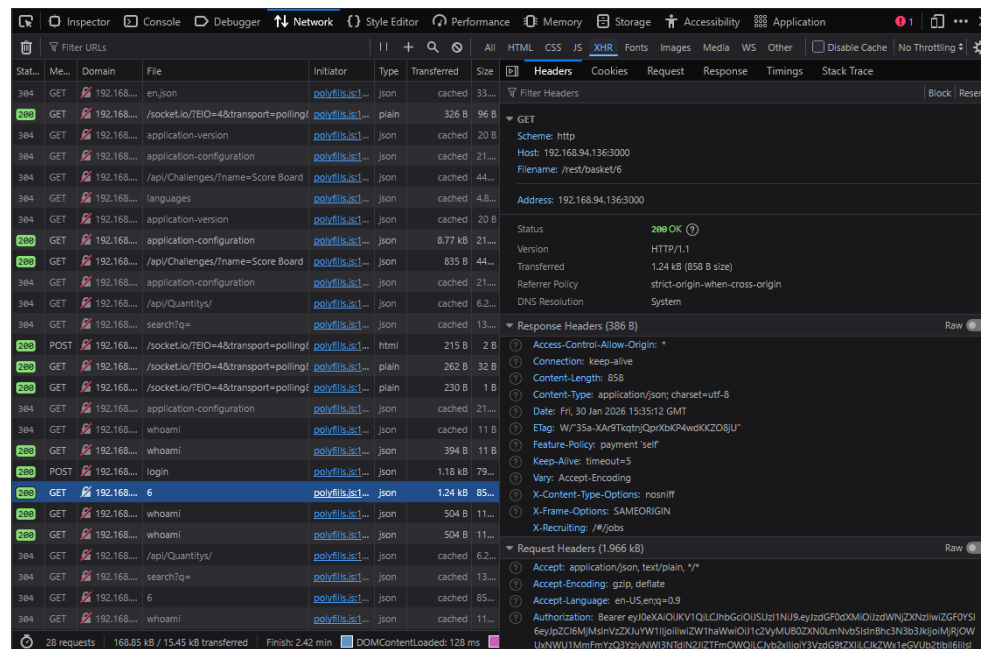
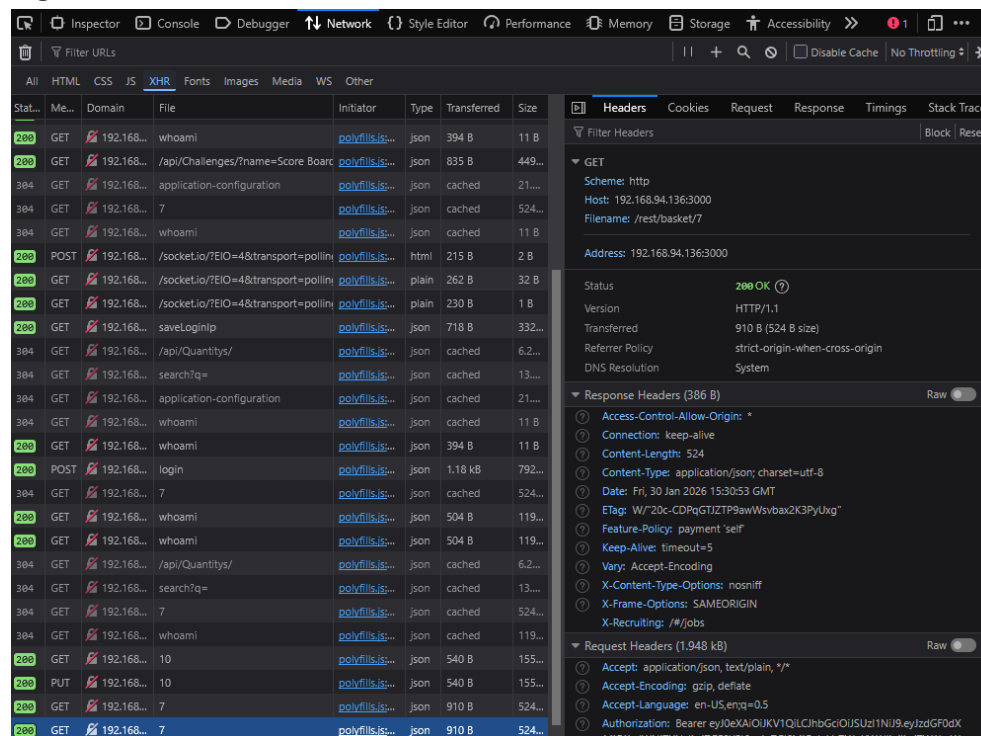


Figure 2



IDOR Validation

Figure 3 & 4: Browser-captured cURL GET requests were manually replayed in PowerShell with authentication preserved, allowing controlled modification of the basket identifier to assess enforcement of object-level authorization and confirm IDOR exposure.

OWASP A01 – Broken Access Control Case Study

Figure 3

```
PS C:\Users\...> $headers = @{"Authorization" = "Bearer ..."}
>> Invoke-WebRequest -Uri "http://192.168...:3000/rest/basket/7" -Headers $headers -Method GET

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
RECOMMENDED ACTION:
    Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?

[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "N"): y

StatusCode      : 200
StatusDescription: OK
Content          : {"status":"success","data":{"id":7,"coupon":null,"UserId":24,"createdAt":"2026-01-29T20:02:21.190Z","updatedAt":"2026-01-29T20:02:21.190Z","Products":[{"id":1
RawContent       : HTTP/1.1 200 OK
                  Access-Control-Allow-Origin: *
                  X-Content-Type-Options: nosniff
                  X-Frame-Options: SAMEORIGIN
                  Feature-Policy: payment 'self'
                  X-Recruiting: /#/jobs
                  Vary: Accept-Encoding
                  Connection:...
Forms            : {}
Headers          : [{"Access-Control-Allow-Origin", "*"}, [{"X-Content-Type-Options", "nosniff"}, [{"X-Frame-Options", "SAMEORIGIN"}, [{"Feature-Policy", "payment 'self'"}]...}
Images           : {}
InputFields      : {}
Links            : {}
ParsedHtml       : mshtml.HTMLDocumentClass
RawContentLength : 524
```

Figure 4

```
PS C:\Users\...> $headers = @{"Authorization" = "Bearer ..."}
>> Invoke-WebRequest -Uri "http://192.168...:3000/rest/basket/6" -Headers $headers -Method GET

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
RECOMMENDED ACTION:
    Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?

[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "N"): y

StatusCode      : 200
StatusDescription: OK
Content          : {"status":"success","data":{"id":6,"coupon":null,"UserId":23,"createdAt":"2026-01-29T20:01:50.177Z","updatedAt":"2026-01-29T20:01:50.177Z","Products":[{"id":4
RawContent       : HTTP/1.1 200 OK
                  Access-Control-Allow-Origin: *
                  X-Content-Type-Options: nosniff
                  X-Frame-Options: SAMEORIGIN
                  Feature-Policy: payment 'self'
                  X-Recruiting: /#/jobs
                  Vary: Accept-Encoding
                  Connection:...
Forms            : {}
Headers          : [{"Access-Control-Allow-Origin", "*"}, [{"X-Content-Type-Options", "nosniff"}, [{"X-Frame-Options", "SAMEORIGIN"}, [{"Feature-Policy", "payment 'self'"}]...}
Images           : {}
InputFields      : {}
Links            : {}
ParsedHtml       : mshtml.HTMLDocumentClass
RawContentLength : 858
```

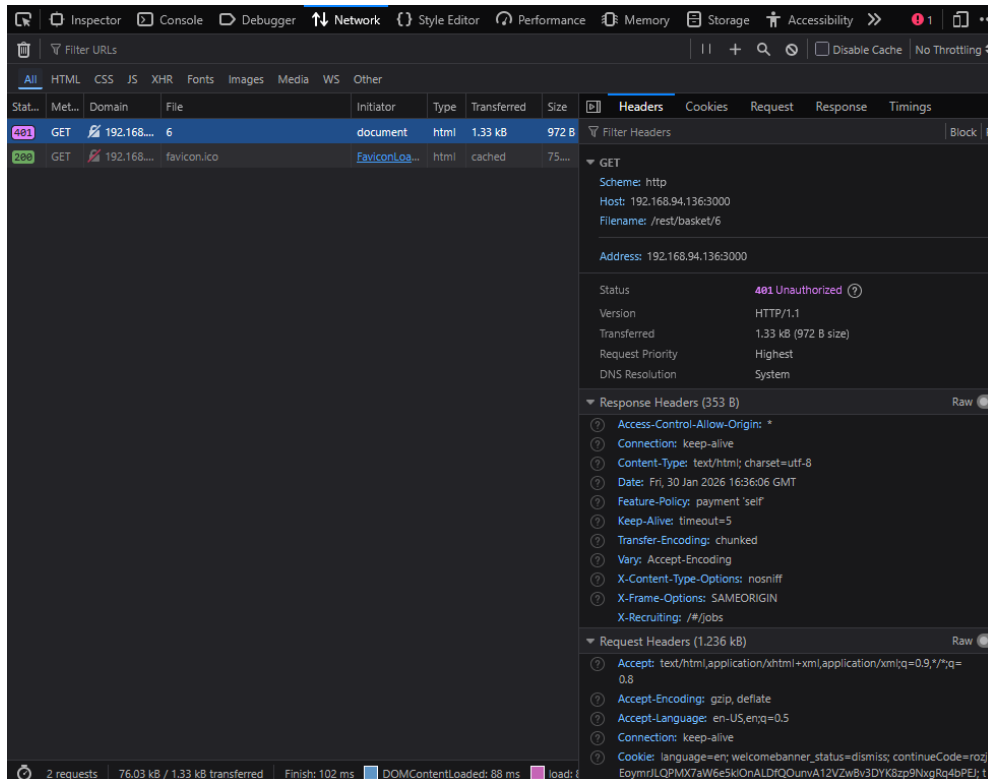
Negative Test: Direct URL Manipulation Attempt

If object-level authorization is enforced, modifying the basket identifier directly in the request URL should be denied for resources not owned by the authenticated user. While authenticated as User 1, the basket identifier in the `/rest/basket/{basketId}` endpoint was manually modified in the browser URL bar to reference a basket owned by User 2.

Figure 6: Direct URL manipulation of the basket identifier resulted in an HTTP `401 Unauthorized` response, indicating authentication enforcement while highlighting inconsistent authorization behavior when compared to authenticated request replay.

OWASP A01 – Broken Access Control Case Study

Figure 6



Remediation

To prevent Insecure Direct Object Reference (IDOR) conditions that allow authenticated users to access or manipulate resources owned by other users, the following remediation steps should be implemented.

Enforce Server-Side Object Ownership Checks

All requests that reference user-owned objects (e.g., baskets) must validate ownership on the server side. The application should ensure that the authenticated user associated with the request is the owner of the referenced resource before returning or modifying data.

Example control:

When handling a request for a basket, the server should verify that ``basket.userId == authenticatedUserId`` before processing the request.

Derive User Context Exclusively From Authentication Data

The application should derive the user identity exclusively from the authentication context (session or JWT) rather than trusting client-supplied identifiers such as user IDs or basket IDs.

OWASP A01 – Broken Access Control Case Study

Client-controlled parameters must *never* be used as the sole basis for authorization decisions.

Scope Database Queries by Authenticated User

Resource queries should be explicitly scoped to the authenticated user to prevent cross-user access.

Example approach:

Instead of querying by basket ID alone, queries should require both the basket ID and the authenticated user's identifier.

This ensures that even valid object identifiers cannot be used to retrieve unauthorized data.

Apply Authorization Checks Consistently Across All Endpoints

Authorization logic must be applied uniformly across all API endpoints that expose user-owned resources, including:

- Read operations (GET)
- Modification operations (POST / PUT / PATCH)
- Destructive operations (DELETE)

Inconsistent enforcement across methods or routes can reintroduce IDOR conditions.

Avoid Predictable or Sequential Object Identifiers

Where feasible, avoid exposing sequential or easily guessable object identifiers. While this does not replace authorization checks, it reduces the likelihood of successful enumeration and abuse.

Authorization must still be enforced regardless of identifier complexity.

Log and Monitor Authorization Failures

Authorization failures related to object access should be logged and monitored to detect potential abuse patterns. Repeated cross-user access attempts may indicate malicious behavior or testing activity that warrants further investigation.

Security Impact(Technical)

Implementing these controls prevents unauthorized access to user data, mitigates privacy and integrity risks, and ensures that authenticated users are restricted to resources explicitly associated with their identity.

Security Impact (Business Context)

In a production environment, this failure could result in privacy violations, loss of customer trust, and regulatory or legal exposure depending on the sensitivity of accessed data.